EstateFinder

A Real-Estate Property Browsing App for Android

End-to-End Project Report & Viva Walkthrough

Platform	**Android (Native)**
Language	**Java**
Architecture	**MVVM + Room + Retrofit**
Package	**com.example.estatefinder**
Min / Target SDK	**API 26 (Android 8.0) / API 34 (Android 14)**
Report type	**Creation → Structure → How it works**

Contents

1. What EstateFinder Is

EstateFinder is a native Android application that lets a user browse, search, filter, sort, and shortlist real-estate property listings (apartments, houses, villas, and offices), view rich detail pages, contact the seller, and share a listing with others.

It is built as an **offline-first** app: every screen reads from a local database, so the app works instantly and never shows a blank screen even with no internet. A mock REST API is available to demonstrate that the same data can be refreshed from a server.

The app is organised around four headline features:

- F1 **Featured Properties** — a curated set highlighted on the home screen with a star badge.
- F2 **Sort Listings** — order results by newest, price, or area.
- F3 **Share Property** — send a formatted summary to any app via the system share sheet.
- F4 **Seller Info & Contact** — call, email, or message the listing's seller directly.

Guiding engineering principle throughout: **stability over feature count**. Every screen is backed by the local database, all database work happens off the main thread, and the UI updates reactively through observable data streams.

2. Creating the Project in Android Studio (Step by Step)

This is exactly how the project skeleton is created before any of our own code is written.

Step 1 — New Project

Open Android Studio → **File** → **New** → **New Project**. On the template chooser, pick **"Empty Views Activity"**. (We choose *Views*, not *Compose*, because this app is built with classic XML layouts + Java.)

Step 2 — Configure the project

Field	Value used
Name	EstateFinder
Package name	`com.example.estatefinder`
Language	Java
Minimum SDK	API 26 (Android 8.0 "Oreo")
Build configuration language	Groovy DSL (`build.gradle`)

Clicking **Finish** makes Android Studio generate a working "Hello World" app.

Step 3 — What the wizard generates for you

Out of the box you get exactly one screen and its supporting files:

- `MainActivity.java` — the single starting Activity (a screen).
- `res/layout/activity_main.xml` — the layout (the visual design) for that screen.
- `AndroidManifest.xml` — the app's "table of contents" that registers every screen and permission.
- `build.gradle` (project + app) — the build scripts and dependency lists.
- `res/values/` — `strings.xml` (text), `colors.xml` (colours), `themes.xml` (styling).

> **Key concept:** In Android, a screen = an **Activity** (the Java logic) + a **layout** (the XML design). Every new screen you want must be (a) written as a Java Activity class, (b) given an XML layout, and (c) registered in the manifest. EstateFinder grows from the single generated `MainActivity` into **six activities** exactly this way.

Step 4 — Building out the rest

From that one generated screen, we then:

1. Add our library dependencies in `app/build.gradle` (Section 3).
2. Create Java **packages** to organise code by responsibility (Section 4).
3. Add new **Activities + layouts** for each screen — Splash, Property List, Details, Favorites, Map (Section 9) — and register each one in the manifest.
4. Create the **Room database**, DTOs, repository and view-model that feed those screens (Sections 6–8).

3. Gradle Setup & Dependencies (and Why Each One)

Gradle is the build system. Two files matter:

Root `build.gradle`

Declares the Android Gradle Plugin version shared by all modules:

```
plugins {
    id 'com.android.application' version '8.7.3' apply false
}
```

Module `app/build.gradle` — core config

```
android {
    namespace 'com.example.estatefinder'
    compileSdk 34
    defaultConfig {
        applicationId "com.example.estatefinder"
        minSdk 26
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
}
```

Dependencies and their jobs

Library	Version	What it does in this app
AppCompat	1.7.0	Backwards-compatible Activity base classes.
Material Components	1.12.0	Material 3 widgets: toolbars, cards, chips, toggle buttons, text fields.
ConstraintLayout	2.2.0	Flexible layout engine used inside screens.
RecyclerView	1.3.2	Efficient scrolling lists of property cards.
Lifecycle ViewModel	2.8.7	Holds UI state so it survives screen rotation.
Lifecycle LiveData	2.8.7	Observable data — the UI auto-updates when data changes.
Room (runtime + compiler)	2.6.1	The local SQLite database layer — our single source of truth.
Retrofit + Gson converter	2.11.0	Type-safe HTTP client to talk to the mock REST API.
OkHttp logging interceptor	4.12.0	Logs network calls during development.
Glide	4.16.0	Loads & caches property images from URLs, with a fallback image.

Deliberately **not** included: Google Maps SDK, Firebase, authentication, payments — kept out to maximise stability and keep the scope tight.

4. The Full Project Structure

Code is grouped into packages by responsibility, so each part of the app has one clear home:

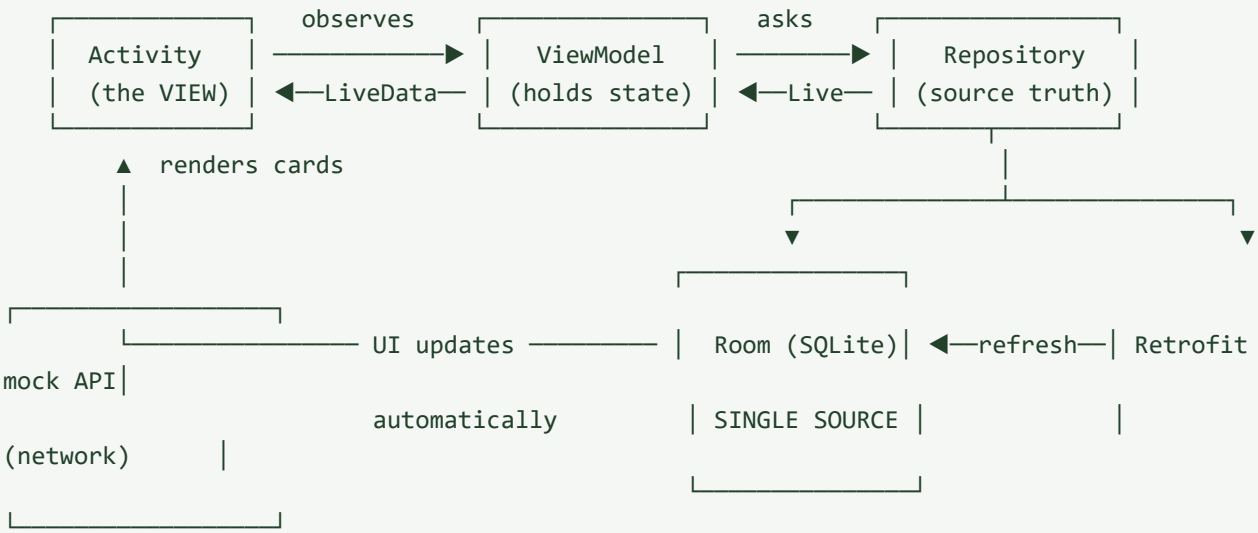
```
com.example.estatefinder
│
├── MainActivity.java                ← Home screen logic
│
├── model/
│   └── Property.java                ← Plain domain object (one listing)
│
├── data/
│   ├── SampleData.java              ← 15 hard-coded seed listings
│   ├── local/                       ← ROOM (local database)
│   │   ├── EstateDatabase.java      ← DB definition, migrations, seeding
│   │   ├── PropertyEntity.java      ← "properties" table row
│   │   ├── FavoriteEntity.java      ← "favorites" table row
│   │   ├── PropertyDao.java         ← Queries for properties
│   │   └── FavoriteDao.java         ← Queries for favorites
│   ├── remote/                      ← RETROFIT (network)
│   │   ├── ApiService.java          ← REST endpoints
│   │   ├── RemoteDataSource.java    ← Retrofit builder + fetch
│   │   ├── dto/PropertyResponse.java ← JSON shape from server
│   │   └── mapper/PropertyMapper.java← DTO ⇄ domain/entity conversion
│   └── repository/
│       └── PropertyRepository.java  ← Single source of truth, merges DB + network
│
├── viewmodel/
│   └── PropertyViewModel.java       ← Holds UI state, exposes LiveData
│
├── util/
│   └── FormatUtils.java             ← Indian-currency price formatting
│
└── ui/
    ├── splash/SplashActivity.java       ← Launch screen
    ├── property/PropertyListActivity.java← Search results list
    ├── property/PropertyAdapter.java    ← RecyclerView adapter (shared)
    ├── details/PropertyDetailsActivity.java ← One property in full
    ├── favorites/FavoritesActivity.java  ← Saved properties
    └── map/MapPlaceholderActivity.java   ← Location fallback screen
```

Matching resources

Type	Files
Layouts (`res/layout`)	activity_splash, activity_main, activity_property_list, activity_property_details, activity_favorites, activity_map_placeholder, **item_property_card** (the reusable list row)
Menus (`res/menu`)	menu_main (refresh), menu_property_list (sort), menu_details (share)
Values (`res/values`)	strings.xml, colors.xml, themes.xml + `values-night/colors.xml` (dark mode)

5. The Architecture: MVVM + Offline-First

The app follows the **MVVM (Model–View–ViewModel)** pattern recommended by Google. Data flows in one clear direction, and the UI simply reacts to changes.



The rules that keep it stable

- **Room is the single source of truth.** The UI *only* ever reads from the database. The network's job is just to *refresh* the database.
- **Everything is reactive.** Screens observe `LiveData`. When the database changes, observers fire and the UI redraws itself — no manual refreshing.
- **No blocking the main thread.** All database writes run on a background executor; reads return `LiveData` that Room populates off-thread. The UI never freezes.
- **State survives rotation.** Because filter/sort state lives in the `ViewModel`, rotating the phone doesn't lose the user's search.

6. The Data Layer (Room)

Room is a wrapper over SQLite. We describe tables as annotated Java classes (*entities*), describe queries as an interface (*DAO*), and Room generates the SQL code.

6.1 Entities (the tables)

`PropertyEntity` — the `properties` table

One row per listing. Key fields: `id` (primary key), `title`, `description`, `price`, `location`, `propertyType` (Apartment/House/Villa/Office), `listingType` (Sale/Rent), `bedrooms`, `bathrooms`, `area`, `latitude`, `longitude`, `image_url`, and the feature-specific fields `featured`, `sellerName`, `sellerPhone`, `sellerEmail`.

`FavoriteEntity` — the `favorites` table

A deliberately tiny table: just `propertyId` as the primary key. A property is a favorite *if and only if* its id appears here. This keeps favorites decoupled from the listing data.

6.2 DAOs (the queries)

`PropertyDao` exposes queries such as:

- `getAll()` / `getFeatured()` (`WHERE featured = 1`) / `getById()` — all returning `LiveData`.
- `search(...)` — filters by title/location text, listing type, and property type.
- Five **separate** sort queries: `searchNewest` (`ORDER BY id DESC`), `searchPriceAsc/Desc`, `searchAreaAsc/Desc`.
- `getFavoriteProperties()` — an `INNER JOIN` between properties and favorites.

> **A deliberate design choice for correctness:** we use five distinct `@Query` methods rather than a dynamic `ORDER BY :param`. SQLite cannot bind a column name as a parameter — doing so silently fails to sort. Separate compile-time-checked queries are safe and predictable.

`FavoriteDao` handles `insert` (ignore on conflict), `delete`, and reading favorite ids.

6.3 The database class, migrations & seeding

`EstateDatabase` is a singleton annotated `@Database(version = 3)` that ties the entities and DAOs together and owns a background write executor.

Migrations — upgrading the schema without losing data

Because a real installed app already holds a user's data, we never wipe the DB on a schema change; we *migrate* it:

Migration	What it does
`MIGRATION_1_2`	Adds the `image_url` column.
`MIGRATION_2_3`	Adds `featured` + the three seller columns, then backfills: marks featured ids, assigns seller triads, and fills image URLs by property type.

Seeding — the app is never empty

On first open, an `onOpen` callback checks `COUNT(*)`; if the table is empty it inserts the 15 sample listings from `SampleData` on the background executor.

> **Why `onOpen`, not `onCreate`?** Seeding inside `onCreate` can re-enter the database while it is still opening and throw *"IllegalStateException"*. Doing it in `onOpen` after the DB is ready avoids that crash — a direct application of *stability first*.

7. The Network Layer (Retrofit + Mock API)

To demonstrate real client–server behaviour without a paid backend, the project ships a **mock API**: a static `properties.json` served over HTTP.

- `ApiService` declares the endpoint: `@GET("properties")` returning a list of `PropertyResponse`.
- `RemoteDataSource` builds Retrofit against `BASE_URL = "http://10.0.2.2:8080/"`. On the Android emulator, `10.0.2.2` is a special alias that loops back to *the host PC's* `localhost` — so the emulator can reach a server running on the developer's machine.
- `PropertyResponse` (a DTO) mirrors the JSON, using `@SerializedName` for fields like `propertyType` and `image_url`.
- `PropertyMapper` converts between the network DTO, the domain `Property`, and the Room entity — so the rest of the app never depends on the JSON shape.

When the user taps **Refresh**, the repository fetches the list and writes it into Room; because the UI observes Room, the screen updates itself. If the network is down, the app simply keeps showing the cached data — offline-first.

8. The Repository & ViewModel (The Reactive Brain)

`PropertyRepository`

The single point that the rest of the app talks to for data. It:

- Exposes `LiveData` streams: `getAllLive()`, `getFeaturedLive()`, `searchLive(query, listing, type, sort)`, `getFavoritePropertiesLive()`.
- Defines the `SortOrder` enum: `NEWEST, PRICE_ASC, PRICE_DESC, AREA_ASC, AREA_DESC`, and picks the matching DAO query.

- Runs writes (favorite toggles, network refresh) on a background executor.
- Maintains a live set of favorite ids so any list can instantly show the correct heart state.

`PropertyViewModel`

An `AndroidViewModel` that holds the current filter state — query text, listing type, property type, and sort order — and exposes the results as a single `MediatorLiveData` called `searchResults`.

When any filter changes, `rebuildSearch()` swaps the underlying source (removing the old source before adding the new one, so sources never stack up) and points `searchResults` at the freshly chosen repository query. The Activity just observes `searchResults` and never worries about *how* the list was produced.

9. Every Screen: Activities & Their Layouts

Six activities, each with its own XML layout. Here is what each one contains and does.

9.1 SplashActivity → `activity_splash.xml`

Layout: a centred logo circle (location icon) over a branded background, with the app name and tagline, animated by a fade-in.

Logic: shows for ~1.4 seconds, then launches `MainActivity` and finishes. This is the app's **launcher** activity (the entry point declared in the manifest).

9.2 MainActivity (Home) → `activity_main.xml`

Layout (a scrollable column):

- Header: app title + a **favorites** icon button (top-right).
- A search field (`TextInputLayout` + `etSearch`).
- A **Buy / Rent** toggle group (All / Buy / Rent).
- A **property-type chip group** (All / Apartment / House / Villa / Office).
- A **Find Properties** button.
- A "Featured Properties" heading + a `RecyclerView` (`rvFeatured`).
- A **View All Properties** button.

Logic: observes the *featured* LiveData and renders those cards; reads the chosen search text/listing/type and opens `PropertyListActivity` with them as intent extras. The menu adds a **Refresh** action.

9.3 PropertyListActivity → `activity_property_list.xml`

Layout: a Material toolbar (with a result-count subtitle), the same search field + Buy/Rent toggle + type chips (chips inside a horizontal scroll view), a full-height `RecyclerView` (`rvProperties`), and an **empty-state** panel shown when nothing matches.

Logic: reads the incoming query/listing/type extras, restores them into the UI controls, and observes the view-model's `searchResults`. The **Sort** menu (a single-choice submenu) maps each option to a `SortOrder` and re-queries. Tapping a card opens the details screen.

9.4 PropertyDetailsActivity → `activity_property_details.xml`

Layout: a large property image (with listing badge + featured star), the price, title, location, three **spec cards** (bedrooms / bathrooms / area), a type & listing line, the description, a **Contact Seller** section (name + Call / Email / Message buttons), and bottom actions (**Favorite** toggle + **View on Map**). The toolbar carries the **Share** action.

Logic: loads the property by id, binds every field, toggles the favorite heart, launches the share sheet, fires the seller-contact intents, and opens the map. If a listing has no seller data, the whole seller section is hidden.

9.5 FavoritesActivity → `activity_favorites.xml`

Layout: toolbar + a `RecyclerView` of saved properties + an empty-state ("No favorites yet").

Logic: observes the favorites join query; the list updates live as the user hearts/un-hearts properties anywhere in the app.

9.6 MapPlaceholderActivity → `activity_map_placeholder.xml`

Layout: a location icon, the property name & location, and its formatted coordinates (`° N, ° E`).

Logic: this is the *fallback* for "View on Map". The details screen first tries to open a real maps app via a `geo:` intent; only if no maps app exists does it fall back to this coordinate-preview screen. (No Google Maps SDK is bundled — a stability/scope decision.)

9.7 The shared row: `item_property_card` + `PropertyAdapter`

All three lists (Home featured, Search results, Favorites) reuse a single `MaterialCardView` row and a single `PropertyAdapter`. Each card shows the image (via Glide with a fallback), a SALE/RENT badge, a favorite heart, an optional featured star, the title, location, price, and specs (e.g. "3 BHK · Apartment"). One adapter, one row layout, three screens — no duplication.

10. The Four Features, End to End

F1 Featured Properties

Data: a `featured` boolean column (set for a curated set of ids during migration/seed). **Query:** `getFeatured()` = `WHERE featured = 1`. **UI:** the home screen observes this list and shows an amber **star** on featured cards. The star reflects the real flag and is independent of Sale/Rent.

F2 Sort Listings

UI: the list toolbar's Sort submenu offers Newest / Price ↑ / Price ↓ / Area ↑ / Area ↓. **Flow:** the selected item maps to a `SortOrder` → the view-model rebuilds `searchResults` → the repository runs the matching compile-time-checked DAO query → the list re-renders reactively. Sorting composes with the active search text and filters.

F3 Share Property

UI: a Share action on the details toolbar. **Flow:** builds a plain-text summary (title, price, location, specs, type, seller) plus a "Shared via EstateFinder" footer, wraps it in an `ACTION_SEND` intent, and presents the system share sheet via `createChooser` — so the user can send it to WhatsApp, email, notes, etc.

F4 Seller Info & Contact

Data: each listing carries `sellerName`, `sellerPhone`, `sellerEmail`. **UI:** a Contact Seller section with Call / Email / Message. **Flow:**

- **Call** → `ACTION_DIAL` with a `tel:` URI (sanitised digits). We use *DIAL*, not *CALL*, so it opens the dialer with the number pre-filled and never places a call without the user tapping — no call permission required.
- **Email** → `mailto:` with a pre-filled subject.
- **Message** → `smsto:` to the SMS app.

Each is wrapped in a try/catch so that if no matching app exists, the user sees a friendly message instead of a crash.

11. Visual Polish (Theme, Day/Night, Animations)

- **Material 3 theme** (`Theme.Material3.DayNight.NoActionBar`) with a green brand palette (primary `#2E7D32`) and an amber accent for featured stars.
- **Automatic dark mode:** a full `values-night/colors.xml` palette. Text colours that were dark-green in light mode become light-green in dark mode so they stay legible; the brand green and amber star stay constant. No layout changes are needed — same colour *names*, different values.
- **Splash screen** with a fade-in animation for a polished launch.
- **List entrance animation:** cards "fall in" via a layout animation when a list appears.
- **Indian-currency formatting** (`FormatUtils`): prices render as `₹32,50,000` and rents as `₹62,000/month` using the Indian digit-grouping style.

12. How It All Works Together (Runtime Walkthrough)

Here is the complete life of a session — the "this is how it works" story:

1. **Launch.** Android starts `SplashActivity` (the launcher). It shows the logo for ~1.4s.

2. **Database opens.** The Room singleton opens; its `onOpen` callback sees the table is empty on first run and seeds the 15 sample listings on a background thread. On later runs, existing data (and any schema upgrade via migrations) is preserved.
3. **Home appears.** `MainActivity` asks the view-model for the featured list. The view-model asks the repository, which returns `LiveData` from Room. When Room delivers the rows, the observer fires and the featured cards render — with images loaded by Glide.
4. **User searches.** They type text, pick Buy/Rent, pick a type, and tap **Find**. `MainActivity` opens `PropertyListActivity`, passing those choices.
5. **Results stream in.** The list screen restores those controls and observes `searchResults`. The repository runs the matching DAO query; the list renders, or the empty-state shows. The toolbar subtitle shows the result count.
6. **User sorts.** Choosing a Sort option rebuilds the query; the same observer re-fires with re-ordered rows. Nothing else in the screen needs to change.
7. **User opens a property.** Tapping a card opens `PropertyDetailsActivity`, which loads that property by id and binds the image, price, specs, description, and seller section.
8. **User acts.** They can **favorite** it (written to the favorites table on a background thread — the heart updates everywhere via LiveData), **share** it (system share sheet), **contact the seller** (dialer/email/SMS), or **view on map** (external maps app, or the coordinate fallback screen).
9. **Favorites.** The favorites icon on Home opens `FavoritesActivity`, which lists exactly the properties whose ids are in the favorites table, live.
10. **Refresh (optional).** The Refresh action pulls fresh data from the mock API into Room; because the UI observes Room, it updates itself. With no network, cached data simply remains — the app never breaks.

Every arrow in that story is the same pattern: **UI observes → ViewModel holds state → Repository reads Room → Room notifies → UI redraws.** That single, repeated, one-directional loop is what makes the whole app predictable and stable.

13. How to Build & Run It

Just the app (offline — recommended for the demo)

1. Open the project in Android Studio and let Gradle sync.
2. Pick an emulator (API 26+) or a real device.
3. Press **Run ▶**. The app seeds itself and works fully offline.

To also demonstrate the network refresh

1. Serve the mock API from the `api-mock/` folder on the host on port 8080, e.g.:

```
cd api-mock
python -m http.server 8080
```

2. Run the app on the **emulator** (so `10.0.2.2` reaches the host) and tap **Refresh**.

14. Viva Talking Points & Conclusion

Strong points to raise in the viva

- **Clean MVVM** with a single source of truth and one-directional, reactive data flow.
- **Offline-first** — the UI always reads from Room; the network only refreshes it.
- **Thread-safe** — no database work on the main thread; the UI never freezes (no ANRs).
- **Robust database** — versioned schema with real migrations, and crash-safe `onOpen` seeding.
- **Correct-by-construction sorting** — separate compile-checked queries instead of an unsafe dynamic `ORDER BY`.
- **Safe external actions** — `ACTION_DIAL` (never auto-calls), and every intent guarded so missing apps don't crash the app.
- **Polished UX** — Material 3, automatic dark mode, animations, Indian-currency formatting, empty states.

Conclusion

EstateFinder is a complete, stable, native Android application that takes a user from a branded splash, through searching, filtering, and sorting a catalogue of property listings, into a rich detail page where they can shortlist, share, and contact a seller — all backed by a local database that keeps the app fast and fully functional offline, with an optional network refresh to prove the client–server path.

It was built by starting from a single generated Activity in Android Studio and growing it — one Activity, one layout, one data class at a time — into a six-screen app organised by the MVVM pattern. The result is deliberately focused: a small, coherent feature set implemented carefully, with **stability chosen over feature count** at every decision. **That is how EstateFinder works.**